

Week 1 TODO List

Oracle and Test Generation for Simplified Jordan Sorting

第一周总目标

先不要急着实现完整 Jordan sorting 主算法。本周目标是建立一个最小可运行闭环：

generate sequence -> oracle checks valid/invalid -> expected sorted order -> save test case -> run baselines

优先级	本周必须产出	用途
P0	src/oracle.py + unit tests	以后验证主算法正确性的基础。
P0	docs/oracle_and_test_generation.md	之后可直接变成 thesis 的一节。
P1	src/generators.py 的简单 valid/invalid generators	解决导师提醒的 test generation 风险。
P1	src/baselines.py	准备 quicksort, merge sort, Python sort 比较。
P2	小规模实验 CSV + week 1 summary	形成可汇报的第一周进展。

Day 1 - 建立项目结构 + 写定义文档

TODO

- ☐ 新建项目目录。
- ☐ 写 docs/oracle_and_test_generation.md。
- ☐ 写清楚 Jordan sequence, upper pairs, lower pairs, intervals, laminarity, crossing, oracle。

```
jordan-sorting/  
  src/  
    oracle.py  
    generators.py  
    baselines.py  
    stats.py  
  tests/  
    test_oracle.py  
    test_generators.py  
  experiments/  
    run_small_tests.py  
  docs/  
    oracle_and_test_generation.md  
    notes.md
```

文档中要写的定义

- **Upper pairs:** $\{x_1, x_2\}, \{x_3, x_4\}, \{x_5, x_6\}, \dots$
- **Lower pairs:** $\{x_2, x_3\}, \{x_4, x_5\}, \{x_6, x_7\}, \dots$
- **Interval:** pair $\{a, b\}$ becomes $[\min(\text{rank}(a), \text{rank}(b)), \max(\text{rank}(a), \text{rank}(b))]$
- **Laminarity:** two intervals are either disjoint or nested.
- **Crossing:** invalid cases are $a < c < b < d$ or $c < a < d < b$.

Day 1 完成标准: 有一份 2-4 页左右的小文档，标题可用 Oracle and Test Instance Generation for Jordan Sorting。

Day 2 - 实现 correctness oracle

在 `src/oracle.py` 中实现

- ☐ `upper_pairs(seq)`
- ☐ `lower_pairs(seq)`
- ☐ `rank_map(seq)`
- ☐ `pair_to_interval(pair, rank)`
- ☐ `crosses(interval1, interval2)`
- ☐ `is_laminar(pairs, rank)` - 第一版可用 $O(n^2)$ ，先保证正确。
- ☐ `oracle(seq)` - 输出 valid/invalid, sorted order, reason。

`oracle(seq)` output example:

```
{
  "valid": true,
  "sorted": [...],
  "upper_ok": true,
  "lower_ok": true,
  "reason": null
}
```

Unit tests

- ☐ flat valid case
- ☐ nested valid case
- ☐ invalid upper crossing
- ☐ invalid lower crossing
- ☐ random permutation sanity check
- ☐ duplicated values rejection, 如果你决定只支持 distinct values

Day 2 完成标准： 运行 `pytest`，oracle 基本测试通过。

Day 3 - 实现最小 test generators

Valid generators

- ☐ `generate_flat(n)` - 例如 `1,2,3,4,5,6,...`，结构浅，主要是 disjoint intervals。
- ☐ `generate_nested(n)` - 例如 `1,n,2,n-1,3,n-2,...`，生成后必须跑 oracle，不要假设一定 valid。
- ☐ `generate_small_handmade_valid_cases()` - 手工小例子，方便 debug 和论文解释。

Invalid generators

- ☐ `generate_invalid_upper_crossing()` - 人为制造 upper interval crossing。
- ☐ `generate_invalid_lower_crossing()` - 人为制造 lower interval crossing。
- ☐ `generate_random_permutation(n)` - 随机 permutation，大概率 invalid，但仍要用 oracle 检查。
- ☐ `mutate_by_swap(seq)` - 从 valid sequence 出发交换两个位置，再用 oracle 确认是否 invalid。

Day 3 完成标准： 你可以调用 flat/nested/random/invalid generators，并用 oracle 正常检查结果。

Day 4 - 设计输入族和保存格式

Input families

- ☐ flat_valid
- ☐ nested_valid
- ☐ random_valid - 第一周可以先写设计，不一定完整实现。
- ☐ split_heavy_valid - 第一周先写思路，不急实现。
- ☐ invalid_crossing
- ☐ random_invalid
- ☐ mutation_based_invalid

JSON 保存格式

```
{
  "id": "flat_n16_001",
  "n": 16,
  "type": "flat_valid",
  "sequence": [1, 2, 3, 4, 5, 6],
  "oracle": {
    "valid": true,
    "sorted": [1, 2, 3, 4, 5, 6],
    "upper_ok": true,
    "lower_ok": true,
    "reason": null
  }
}
```

实现

- ☐ save_test_case(seq, family, path)
- ☐ load_test_case(path)
- ☐ generate_dataset(family, sizes, repetitions)

Day 4 完成标准： 有一个小型 data/ 目录，包含 flat, nested, invalid, random invalid 的 JSON 测试用例。

Day 5 - Baselines skeleton

在 `src/baselines.py` 中实现

- ☐ `python_sort(seq)` - 使用 `sorted(seq)`。
- ☐ `merge_sort(seq)` - 自己写一个简单版本。
- ☐ `quick_sort(seq)` - 自己写一个简单版本，先作为 baseline，不急着优化。
- ☐ `sort_plus_laminarity_check(seq)` - 最重要 baseline: Python sort + upper/lower laminarity oracle。
- ☐ `time_function(func, seq)` - 简单记录运行时间。

```
baseline result example:
{
  "algorithm": "python_sort",
  "n": 1024,
  "time_ms": 0.12
}
```

Day 5 完成标准： 对同一批 test cases 能跑 Python sort, merge sort, quicksort, sort + laminarity check。

Day 6 - 小规模实验跑通

实现 `experiments/run_small_tests.py`

- ☐ 设置 sizes: [8, 16, 32, 64, 128, 256, 512]
- ☐ 设置 families: flat_valid, nested_valid, invalid_crossing, random_invalid
- ☐ 每组 repetitions: 10
- ☐ 记录 oracle valid/invalid 结果。
- ☐ 记录 Python sort, merge sort, quicksort, sort + laminarity check 的时间。
- ☐ 输出 `results/week1_baseline_results.csv`。

```
CSV columns:
run_id,n,family,valid,algorithm,time_ms
```

Day 6 完成标准： 有第一份实验 CSV；不用急着画图，但至少能看到表格数据。

Day 7 - 写一页总结 + 整理问题

写 `docs/week1_summary.md`

- ☐ **What I implemented:** oracle, laminarity check, generators, baselines, small experiment runner.
- ☐ **What works:** oracle 能识别简单 valid/invalid examples; generator 生成的 case 会被自动验证。
- ☐ **Open issues:** random valid generation 不简单; split-heavy valid generation 需要更仔细构造; 需要保证 generated valid cases 结构多样。
- ☐ **Next steps:** 改进 random valid generator; 设计 split-heavy generator; 开始实现 simplified Jordan-sorting framework; 加入 instrumentation counters。

Day 7 完成标准： 有一页清楚的 week 1 progress note，可作为下次 meeting 材料。

本周不要做

- ☐ 不要实现 heterogeneous finger tree。
- ☐ 不要实现 level-linked 2-4 tree。
- ☐ 不要急着写完整 Introduction。
- ☐ 不要急着做大规模实验。
- ☐ 不要急着优化性能。
- ☐ 不要直接实现完整 Jordan sorting 主算法。

如果时间不够，最低完成这 4 个

最低交付物	完成标准
<code>oracle.py</code>	能判断 valid/invalid，并返回 expected sorted order。
<code>test_oracle.py</code>	有 flat, nested, crossing 等基本 unit tests。
<code>generators.py</code>	至少有 flat, nested, invalid crossing generators。
<code>oracle_and_test_generation.md</code>	清楚写出 oracle 和 test generation 的设计。

第一周成功标准：
你不需要完成主算法。只要 oracle + test generation + baseline skeleton 跑通，就已经是正确开局。